

The DVI-format and the program DVItypE

Ernst Reissner (rei3ner@arcor.de)

2025-06-28

Contents

1	Introduction	1
2	The Op-Codes	2
3	The Preamble	5
4	Font definitions	5
5	The Postamble	6
6	References	6

1 Introduction

This document describes the DVI file format traditionally used by L^AT_EX and still in use with `htlatex` and that like.

This self written document is strictly speaking obsolete: The ultimate reference is [Fuc95]. Nevertheless, this document may serve as a description of a very simple subset of the language.

A DVI file is a stream of 8-bit bytes, which may be regarded as a series of commands in a machine-like language. The first byte of each command is the operation code, and this code is followed by zero or more bytes that provide parameters to the command. The parameters themselves may consist of several consecutive bytes; for example, the ‘`set_rule`’ command has two parameters, each of which is four bytes long. Parameters are usually regarded as non-negative integers; but four-byte-long parameters, and shorter parameters that denote distances, can be either positive or negative. Such parameters are given in two’s complement notation. For example, a two-byte-long distance parameter has a value between -2^{15} and $2^{15} - 1$.

A DVI file consists of a “preamble,” followed by a sequence of one or more “pages,” followed by a “postamble”. The preamble is simply a `pre` command, with its parameters that define the dimensions used in the file; this must come first. Each “page” consists of a `bop` command, followed by any number of other commands that tell where characters are to be placed on a physical page, followed by an `eop` command. The pages appear in the order that they were generated, not in any particular numerical order. If we ignore `nop` commands and `fnt_def` commands (which are allowed between any two commands in the file), each `eop` command is immediately followed by a `bop` command, or by a `post` command; in the latter case, there are no more pages in the file, and the remaining bytes form the postamble. Further details about the postamble will be explained later.

Some parameters in DVI commands are “pointers”. These are four-byte quantities that give the location number of some other byte in the file; the first byte is number 0, then comes number 1, and so on. For example, one of the parameters of a **bop** command points to the previous **bop**; this makes it feasible to read the pages in backwards order, in case the results are being directed to a device that stacks its output face up. Suppose the preamble of a DVI file occupies bytes 0 to 99. Now if the first page occupies bytes 100 to 999, say, and if the second page occupies bytes 1000 to 1999, then the **bop** that starts in byte 1000 points to 100 and the **bop** that starts in byte 2000 points to 1000. (The very first **bop**, i.e., the one that starts in byte 100, has a pointer of -1 .)

The DVI format is intended to be both compact and easily interpreted by a machine. Compactness is achieved by making most of the information implicit instead of explicit. When a DVI-reading program reads the commands for a page, it keeps track of several quantities:

- The current font f is an integer; this value is changed only by **fnt** and **fnt_num** commands.
- The current position on the page is given by two numbers called the horizontal and vertical coordinates, h and v . Both coordinates are zero in the upper left corner of the page; moving to the right corresponds to increasing the horizontal coordinate, and moving down corresponds to increasing the vertical coordinate. Thus, the coordinates are essentially Cartesian, except that vertical directions are flipped; the Cartesian version of (h, v) would be $(h, -v)$.
- The current spacing amounts are given by four numbers w, x, y , and z , where w and x are used for horizontal spacing and where y and z are used for vertical spacing.
- There is a stack containing (h, v, w, x, y, z) values; the DVI commands **push** and **pop** are used to change the current level of operation. Note that the current font f is not pushed and popped; the stack contains only information about positioning.

The values of h, v, w, x, y , and z are signed integers having up to 32 bits, including the sign. Since they represent physical distances, there is a small unit of measurement such that increasing h by 1 means moving a certain tiny distance to the right. The actual unit of measurement is variable, as explained below.

Here is a list of all the commands that may appear in a DVI file. Each command is specified by its symbolic name (e.g., **bop**), its opcode byte (e.g., 139), and its parameters (if any). The parameters are followed by a bracketed number telling how many bytes they occupy; for example, ‘ $p[4]$ ’ means that parameter p is four bytes long.

2 The Op-Codes

0–127 **set_char_** $\langle d \rangle$ d where $d \in \{0, \dots, 127\}$.

Typeset character number d from font f such that the reference point of the character is at (h, v) . Then increase h by the width of that character. Note that a character may have zero or negative width, so one cannot be sure that h will advance after this command; but h usually does increase.

128–131 **set_** $\langle d \rangle$ $128 + d$ $c[d]$ where $d \in \{1, \dots, 4\}$. Same as **set_char_0**, except that character number c is typeset. Note that d determines the number of characters. **set_2** is intended for oriental languages with a script with a lot of characters.

132 **setrule** 132 $a[4]$ $b[4]$. Typeset a solid black rectangle of height a and width b , with its bottom left corner at (h, v) . Then set $h := h + b$. If either $a \leq 0$ or $b \leq 0$, nothing

should be typeset. Note that if $b < 0$, the value of h will decrease even though nothing else happens. Programs that typeset from DVI files should be careful to make the rules line up carefully with digitized characters, as explained in connection with the `rule_pixels` subroutine below.

137 `put` $\langle d \rangle$ 132 + d $c[d]$ where $d \in \{1, \dots, 4\}$. Typeset character number c from font f such that the reference point of the character is at (h, v) . (The ‘put’ commands are exactly like the ‘set’ commands, except that they simply put out a character or a rule without moving the reference point afterwards.)

137 `put_rule` 137 $a[4]$ $b[4]$. Same as `set_rule`, except that h is not changed.

138 `nop`. No operation, do nothing. Any number of `nop`’s may occur between DVI commands, but a `nop` cannot be inserted between a command and its parameters or between two parameters.

139 `bop` 139 $c_0[4]$ $c_1[4]$... $c_9[4]$ $p[4]$. Indicates the beginning of a page: Set $(h, v, w, x, y, z) := (0, 0, 0, 0, 0, 0)$ and set the stack empty. Set the current font f to an undefined value. The ten c_i parameters can be used to identify pages, if a user wants to print only part of a DVI file; $\text{\TeX}82$ gives them the values of `count0` ... `count9` at the time `shipout` was invoked for this page. The parameter p points to the previous `bop` command in the file, where the first `bop` has $p = -1$.

140 `eop` 140 End of page: Print what you have read since the previous `bop`. At this point the stack should be empty. (The DVI-reading programs that drive most output devices will have kept a buffer of the material that appears on the page that has just ended. This material is largely, but not entirely, in order by v coordinate and (for fixed v) by h coordinate; so it usually needs to be sorted into some order that is appropriate for the device in question. `DVItyp` does not do such sorting.)

141 `push` 141. Push the current values of (h, v, w, x, y, z) onto the top of the stack; do not change any of these values. Note that f is not pushed.

142 `pop` 142. Pop the top six values off of the stack and assign them to (h, v, w, x, y, z) . The number of pops should never exceed the number of pushes, since it would be highly embarrassing if the stack were empty at the time of a `pop` command.

143–146 `right` $\langle d \rangle$ 142 + d $b[d]$ where $d \in \{1, \dots, 4\}$. Set $h := h + b$, i.e., move right b units. The parameter is a signed number in two’s complement notation; if $b < 0$, the reference point actually moves left.

147 `w0` 147. Set $h := h + w$; i.e., move right w units. With luck, this parameter-less command will usually suffice, because the same kind of motion will occur several times in succession; the following commands explain how w gets particular values.

148–151 `w` $\langle d \rangle$ 147 + d $b[d]$ where $d \in \{1, \dots, 4\}$. Set $w := b$ and $h := h + b$. The value of b is a signed quantity in two’s complement notation. This command changes the current w spacing and moves right by b .

152 `x0` 152. Set $h := h + x$; i.e., move right x units. The ‘ x ’ commands are like the ‘ w ’ commands except that they involve x instead of w .

- 153–156 **x**< d > $152 + d$ $b[d]$ where $d \in \{1, \dots, 4\}$. Set $x := b$ and $h := h + b$. The value of b is a signed quantity in two's complement notation. This command changes the current x spacing and moves right by b .
- 159 **down**< d > $156 + d$ $b[d]$ where $d \in \{1, \dots, 4\}$. Set $v := v + a$, i.e., move down a units. The parameter is a signed number in two's complement notation; if $a < 0$, the reference point actually moves up.
- 161 **y0** 161. Set $v := v + y$; i.e., move down y units. With luck, this parameter-less command will usually suffice, because the same kind of motion will occur several times in succession.
- 162–165 **y**< d > $161 + d$ $a[d]$ where $d \in \{1, \dots, 4\}$. Set $y := a$ and $v := v + a$. The value of a is a signed quantity in two's complement notation. This command changes the current y spacing and moves down by a .
- 166 **z0** 166. Set $v := v + z$; i.e., move down z units. With luck, this parameter-less command will usually suffice, because the same kind of motion will occur several times in succession. The ‘**z**’ commands are like the ‘**y**’ commands except that they involve z instead of y .
- 167–170 **z**< d > $166 + d$ $a[d]$ where $d \in \{1, \dots, 4\}$. Set $y := a$ and $v := v + a$. The value of a is a signed quantity in two's complement notation. This command changes the current z spacing and moves down by a .
- 171 **fntnum0** 171. Set $f := 0$. Font 0 must previously have been defined by a **fnt_def** instruction, as explained below.
- 172–234 **fntnum**< d > $171 + d$ where $d \in \{1, \dots, 63\}$. Set $f := d$.
- 235–238 **fnt**< d > $235 + d$ $k[d]$ where $d \in \{1, \dots, 4\}$. Set $f := k$. An application of **fnt2** or higher maybe used for fonts with colors and texture information.
- 239–242 **xxx**< d > $238 + d$ $k[2]$ $x[k]$ where $d \in \{1, \dots, 4\}$. This command is undefined in general; it functions as a $(k + 2)$ -byte **nop** unless special DVI-reading programs are being used. **T_EX82** generates **xxx1** when a short enough **special** appears, setting k to the number of bytes being sent. It is recommended that x be a string having the form of a keyword followed by possible parameters relevant to that keyword.
- 243–246 **fntdef**< d > $242 + d$ $k[d]$ $c[4]$ $s[4]$ $d[4]$ $a[1]$ $l[1]$ $n[a + l]$ where $d \in \{1, \dots, 4\}$. Define font k ; font definitions will be explained shortly.
- 247 **pre** 247 $i[1]$ $num[4]$ $den[4]$ $mag[4]$ $k[1]$ $x[k]$. Beginning of the preamble; this must come at the very beginning of the file. Parameters i , num , den , mag , k , and x are explained below.
- 248 **post** 248. Beginning of the postamble, see below.
- 249 **post_post** 249. Ending of the postamble, see below.
- 250–255 undefined currently.

3 The Preamble

The preamble contains basic information about the file as a whole. As stated above, there are six parameters:

$$i[1] \mid \text{num}[4], \text{den}[4], \text{mag}[4], k[1], x[k]$$

The i byte identifies DVI format; currently this byte is always set to 2. (The value $i = 3$ is currently used for an extended format that allows a mixture of right-to-left and left-to-right typesetting. Some day we will set $i = 4$, when DVI format makes another incompatible change—perhaps in the year 2048.)

The next two parameters, num and den , are positive integers that define the units of measurement; they are the numerator and denominator of a fraction by which all dimensions in the DVI file could be multiplied in order to get lengths in units of 10^{-7} meters. (For example, there are exactly 7227 T_EX points in 254 centimeters, and T_EX82 works with scaled points where there are 2^{16} sp in a point, so T_EX82 sets $\text{num} = 25400000$ and $\text{den} = 7227 \cdot 2^{16} = 473628672$.)

The mag parameter is what T_EX82 calls **mag**, i.e., 1000 times the desired magnification. The actual fraction by which dimensions are multiplied is therefore $mn/1000d$. Note that if a T_EX source document does not call for any ‘true’ dimensions, and if you change it only by specifying a different **mag** setting, the DVI file that T_EX creates will be completely unchanged except for the value of mag in the preamble and postamble. (Fancy DVI-reading programs allow users to override the mag setting when a DVI file is being printed.)

Finally, k and x allow the DVI writer to include a comment, which is not interpreted further. The length of comment x is k , where $0 \leq k < 256$.

4 Font definitions

Font definitions for a given font number k contain further parameters

$$c[4] \mid s[4] \mid d[4] \mid a[1] \mid l[1] \mid n[a + l].$$

The four-byte value c is the check sum that T_EX (or whatever program generated the DVI file) found in the TFM file for this font; c should match the check sum of the font found by programs that read this DVI file.

Parameter s contains a fixed-point scale factor that is applied to the character widths in font k ; font dimensions in TFM files and other font files are relative to this quantity, which is always positive and less than 2^{27} . It is given in the same units as the other dimensions of the DVI file. Parameter d is similar to s ; it is the “design size” and (like s) it is given in DVI units. Thus, font k is to be used at $\text{mag} \cdot s/1000d$ times its normal size.

The remaining part of a font definition gives the external name of the font, which is an ASCII string of length $a + l$. The number a is the length of the “area” or directory, and l is the length of the font name itself; the standard local system font area is supposed to be used when $a = 0$. The n field contains the area in its first a bytes.

Font definitions must appear before the first use of a particular font number. Once font k is defined, it must not be defined again; however, we shall see below that font definitions appear in the postamble as well as in the pages, so in this sense each font number is defined exactly twice, if at all. Like **nop** commands, font definitions can appear before the first **bop**, or between an **eop** and a **bop**.

5 The Postamble

The last page in a DVI file is followed by ‘`post`’; this command introduces the postamble, which summarizes important facts that `TEX` has accumulated about the file, making it possible to print subsets of the data with reasonable efficiency. The postamble has the form Here p is a pointer to the final `bop` in the file. The next three parameters, *num*, *den*, and *mag*, are duplicates of the quantities that appeared in the preamble.

Parameters l and u give respectively the height-plus-depth of the tallest page and the width of the widest page, in the same units as other dimensions of the file. These numbers might be used by a DVI-reading program to position individual “pages” on large sheets of film or paper; however, the standard convention for output on normal size paper is to position each page so that the upper left-hand corner is exactly one inch from the left and the top. Experience has shown that it is unwise to design DVI-to-printer software that attempts cleverly to center the output; a fixed position of the upper left corner is easiest for users to understand and to work with. Therefore, l and u are often ignored.

Parameter s is the maximum stack depth (i.e., the largest excess of `push` commands over `pop` commands) needed to process this file. Then comes t , the total number of pages (`bop` commands) present.

The postamble continues with font definitions, which are any number of `fnt_def` commands as described above, possibly interspersed with `nop` commands. Each font number that is used in the DVI file must be defined exactly twice: Once before it is first selected by a `fnt` command, and once in the postamble.

@ The last part of the postamble, following the `post_post` byte that signifies the end of the font definitions, contains q , a pointer to the `post` command that started the postamble. An identification byte, i , comes next; this currently equals 2, as in the preamble.

The i byte is followed by four or more bytes that are all equal to the decimal number 223 (i.e., @’337 in octal). `TEX` puts out four to seven of these trailing bytes, until the total length of the file is a multiple of four bytes, since this works out best on machines that pack four bytes per word; but any number of 223’s is allowed, as long as there are at least four of them. In effect, 223 is a sort of signature that is added at the very end.

This curious way to finish off a DVI file makes it feasible for DVI-reading programs to find the postamble first, on most computers, even though `TEX` wants to write the postamble last. Most operating systems permit random access to individual words or bytes of a file, so the DVI reader can start at the end and skip backwards over the 223’s until finding the identification byte. Then it can back up four bytes, read q , and move to byte q of the file. This byte should, of course, contain the value 248 (`post`); now the postamble can be read, so the DVI reader discovers all the information needed for typesetting the pages. Note that it is also possible to skip through the DVI file at reasonably high speed to locate a particular page, if that proves desirable. This saves a lot of time, since DVI files used in production jobs tend to be large.

<ftp://ftp.rrzn.uni-hannover.de/pub/mirror/tex-archive/systems/knuth/dist/texware/dvitype.web>

6 References

[Fuc95] The DVItyp processor. texdoc dvitype, 12 1995. Version 3.6.